

ISTIO SERVICE MESH WITH API GATEWAY

API Gateway → VPC Link → Istio Ingress Gateway → EKS Pods

Complete Setup Guide with Architecture, Configuration & Best Practices

Load Balancing • Circuit Breaking • mTLS • Canary Deployments • Observability

Interview & System Design Preparation

TABLE OF CONTENTS

1. What is Istio?
2. Complete Architecture
3. Component Responsibilities
4. Step-by-Step Setup
5. Request Flow
6. Istio Features: Load Balancing
7. Istio Features: Circuit Breaker
8. Istio Features: Canary Deployments
9. Istio Features: Mutual TLS (mTLS)
10. Istio Features: Retries & Timeouts
11. Istio vs ALB Feature Comparison
12. Observability Stack
13. Resource Overhead
14. When to Use This Pattern
15. Interview-Ready Answers

1. What is Istio?

Istio is a service mesh — a dedicated infrastructure layer that handles service-to-service communication. It injects a sidecar proxy (Envoy) next to every Pod. All traffic flows through this proxy, giving you load balancing, circuit breaking, mTLS, retries, and observability without any code changes.

Without Istio:

```
Pod A ----- direct call -----> Pod B
(no control over the connection)
```

With Istio:

```
Pod A -> [Envoy Proxy] -> [Envoy Proxy] -> Pod B
        (sidecar A)       (sidecar B)
```

Istio controls EVERYTHING in between:

Load balancing, encryption, retries, circuit breaking

Pod Structure with Istio

Before Istio injection:

```
Pod: [ Your App Container ]
```

After Istio injection (automatic):

```
Pod: [ Your App Container ] + [ Envoy Proxy Sidecar ]
                               (auto-injected)
```

Your app talks on localhost:8080

Envoy intercepts ALL inbound/outbound traffic

Your app has ZERO awareness of Envoy

2. Complete Architecture



3. Component Responsibilities

Component	Responsibility
API Gateway	External auth, rate limiting, API keys, throttling
VPC Link	Private tunnel from API Gateway into VPC
Istio Ingress Gateway	Replaces ALB — routes external traffic into the mesh
Envoy Sidecar	Per-pod proxy: load balancing, retries, circuit breaking, mTLS
Istiod (Control Plane)	Configures all Envoy proxies, manages certificates and policies

4. Step-by-Step Setup

Step 1: Install Istio in EKS

```
# Install istioctl
curl -L https://istio.io/downloadIstio | sh -
cd istio-1.20.0
export PATH=$PWD/bin:$PATH

# Install Istio with default profile
istioctl install --set profile=default -y

# Output:
# Istio core installed
# Istiod installed      <- control plane
# Ingress gateway installed <- replaces ALB
```

Step 2: Enable Sidecar Injection

```
# Label namespace - Istio auto-injects Envoy sidecar
kubectl label namespace default istio-injection=enabled

# Every Pod deployed in this namespace now gets:
#   Your container + Envoy sidecar (automatically)
```

Step 3: Deploy Your Application

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
spec:
  replicas: 3
  selector:
    matchLabels:
      app: order-service
  template:
    metadata:
      labels:
        app: order-service
    spec:
      containers:
        - name: order-service
          image: 123456.dkr.ecr.../order-service:latest
          ports:
            - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: order-service
spec:
  selector:
    app: order-service
  ports:
    - port: 80
      targetPort: 8080
```

Step 4: Configure Istio Gateway

```
# gateway.yaml - defines HOW traffic enters the mesh
apiVersion: networking.istio.io/v1beta1
kind: Gateway
metadata:
  name: my-gateway
spec:
  selector:
    istio: ingressgateway
  servers:
    - port:
        number: 80
        name: http
        protocol: HTTP
      hosts:
        - 'api.myapp.com'
```

Step 5: Configure VirtualService (Routing Rules)

```
# virtual-service.yaml - defines WHERE traffic goes
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: my-routes
spec:
  hosts:
    - 'api.myapp.com'
  gateways:
    - my-gateway
  http:
    - match:
        - uri:
            prefix: /api/orders
          route:
            - destination:
                host: order-service
                port:
                  number: 80
        - match:
            - uri:
                prefix: /api/payments
          route:
            - destination:
                host: payment-service
                port:
                  number: 80
```

Step 6: Expose Istio Ingress as Internal NLB

```
# Patch Istio ingress to use internal NLB (not public)
apiVersion: v1
kind: Service
metadata:
  name: istio-ingressgateway
  namespace: istio-system
  annotations:
    service.beta.kubernetes.io/aws-load-balancer-internal: 'true'
    service.beta.kubernetes.io/aws-load-balancer-type: 'nlb'
spec:
  type: LoadBalancer
```

Note: This creates an internal NLB (not accessible from internet). API Gateway connects to it via VPC Link. The NLB is created automatically by Kubernetes — you do not provision it manually.

Step 7: Create VPC Link and Connect API Gateway

VPC Link v2:

Points to the internal NLB created by Istio

API Gateway HTTP API:

Route: ANY /api/{proxy+}

Integration: VPC Link -> NLB -> Istio Ingress Gateway

5. Request Flow

1. User: GET `https://api.myapp.com/api/orders/123`
2. Route 53 -> API Gateway
API GW checks: API key, rate limit, JWT
3. API Gateway -> VPC Link -> NLB (internal)
4. NLB -> Istio Ingress Gateway
Gateway checks: host matches 'api.myapp.com'
5. Istio Ingress -> VirtualService routing
`/api/orders` -> `order-service`
6. Envoy sidecar of `order-service` Pod
Load balances across 3 `order-service` Pods
Picks healthiest Pod using active health checks
7. `Order-service` processes request -> returns response
8. Response path:
Pod -> Envoy -> Istio Ingress -> NLB -> VPC Link
-> API GW -> User

6. Istio Feature: Load Balancing

```
# destination-rule.yaml
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: order-service-lb
spec:
  host: order-service
  trafficPolicy:
    loadBalancer:
      simple: ROUND_ROBIN
```

Algorithm	How It Works
ROUND_ROBIN	Equal distribution across Pods (default)
LEAST_CONN	Send to Pod with fewest active connections
RANDOM	Random Pod selection
PASSTHROUGH	No balancing, direct connection

7. Istio Feature: Circuit Breaker

```
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: order-service-cb
spec:
  host: order-service
  trafficPolicy:
    connectionPool:
      tcp:
        maxConnections: 100
      http:
        http1MaxPendingRequests: 100
        http2MaxRequests: 1000
    outlierDetection:
      consecutive5xxErrors: 3      # 3 failures ->
      interval: 30s               # check every 30s
      baseEjectionTime: 60s       # eject for 60s
      maxEjectionPercent: 50      # max 50% ejected
```

How Outlier Detection Works

```
Pod A: 200, 200, 200      -> HEALTHY
Pod B: 500, 500, 500      -> 3 consecutive 5xx -> EJECTED
Pod C: 200, 200, 200      -> HEALTHY
```

Traffic goes to A and C only.
After 60s, Pod B is retried.
If healthy -> back in rotation.

8. Istio Feature: Canary Deployments

Traffic Splitting — Route a percentage of traffic to the new version. Gradually shift from 90/10 to 0/100. Roll back instantly if errors detected.

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: order-service-canary
spec:
  hosts:
    - order-service
  http:
    - route:
        - destination:
            host: order-service
            subset: v1
            weight: 90          # 90% to old version
        - destination:
            host: order-service
            subset: v2
            weight: 10         # 10% to new version
    ---
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: order-service-versions
spec:
  host: order-service
  subsets:
    - name: v1
      labels:
        version: v1
    - name: v2
      labels:
        version: v2
```

Traffic flow:
100 requests arrive
-> 90 go to v1 Pods
-> 10 go to v2 Pods (canary)

Gradual rollout:
90/10 -> 80/20 -> 50/50 -> 0/100
If v2 has errors -> shift back to 100/0 instantly

9. Istio Feature: Mutual TLS (mTLS)

```
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: default
  namespace: default
spec:
  mtls:
    mode: STRICT    # all traffic encrypted
```

Without Istio mTLS:

Pod A ---- plain HTTP ----> Pod B
Anyone in the network can sniff traffic

With Istio mTLS:

Pod A -> Envoy == encrypted TLS == Envoy -> Pod B
Certificates auto-rotated by Istio
Zero code changes in your app

Key benefit: mTLS is completely transparent to your application. Your app still talks plain HTTP on localhost. Envoy handles all encryption/decryption. Certificates are automatically provisioned and rotated by Istiod.

10. Istio Feature: Retries & Timeouts

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: order-service
spec:
  hosts:
    - order-service
  http:
    - route:
        - destination:
            host: order-service
      timeout: 5s          # max wait time
      retries:
        attempts: 3        # retry 3 times
        perTryTimeout: 2s  # each attempt max 2s
        retryOn: 5xx,reset # retry on server errors
```

No code changes needed. Without Istio, you would implement retries using Resilience4j or Spring Retry in every microservice. With Istio, retries are configured in YAML and applied by the Envoy sidecar automatically.

11. Istio vs ALB Feature Comparison

Feature	ALB	Istio
Load balancing	Round-robin, least conn	Round-robin, least conn, random, consistent hash
Health checks	HTTP/TCP periodic	Active + passive (outlier detection)
Circuit breaker	No (need Resilience4j)	Built-in ■
Canary deployments	Weighted target groups	Traffic splitting ■
mTLS	No	Automatic ■
Retries	No	Built-in ■
Timeouts	Idle timeout only	Per-route configurable ■
Rate limiting	No (need API Gateway)	Built-in ■
Observability	CloudWatch	Jaeger, Kiali, Prometheus, Grafana
Path-based routing	Listener rules	VirtualService ■
Fault injection	No	Built-in ■
Connection draining	Yes	Yes ■
Cost	~\$20/month + LCU	Free (open source) but CPU/memory overhead
Complexity	Low	High

12. Observability Stack

Istio integrates with a full observability stack out of the box. These tools provide visibility into every service-to-service call in the mesh.

Tool	Purpose	What It Shows
Kiali	Service mesh dashboard	Visualize service-to-service traffic, health status
Jaeger	Distributed tracing	Trace requests across 10+ microservices, find latency
Prometheus	Metrics collection	Request rate, error rate, latency (RED metrics)
Grafana	Dashboards	Visualize all metrics, alerting on error rates

Installation

```
kubectl apply -f samples/addons/kiali.yaml
kubectl apply -f samples/addons/jaeger.yaml
kubectl apply -f samples/addons/prometheus.yaml
kubectl apply -f samples/addons/grafana.yaml

# Access Kiali dashboard
istioctl dashboard kiali
```

13. Resource Overhead

Each Pod gets an Envoy sidecar that consumes additional CPU and memory. The control plane (Istiod) also requires resources. Plan your cluster capacity accordingly.

Component	CPU	Memory
Envoy sidecar (per Pod)	~100m (0.1 vCPU)	~50-100 MB
Istiod (control plane)	~500m	~1 GB

Overhead by Cluster Size

Cluster Size	Extra CPU	Extra Memory	Est. Extra Cost/Month
10 Pods	~1.5 vCPU	~2 GB	~\$20-30
50 Pods	~5.5 vCPU	~6 GB	~\$50-100
100 Pods	~10.5 vCPU	~11 GB	~\$100-200

14. When to Use This Pattern

Use Istio + API Gateway When:

- ■ 20+ microservices (mesh benefits scale with service count)
- ■ Need mTLS between all services (compliance, security)
- ■ Need canary deployments with traffic splitting
- ■ Need circuit breaking without code changes
- ■ Need distributed tracing across services
- ■ Team has Kubernetes expertise
- ■ Multi-language microservices (Istio is language-agnostic)

Do NOT Use When:

- ■ Fewer than 10 microservices (overhead not worth it)
- ■ Team is new to Kubernetes
- ■ Simple request routing is sufficient (ALB is enough)
- ■ Resource-constrained cluster (sidecars use CPU/memory)
- ■ Already using AWS App Mesh (similar functionality, AWS-native)

15. Interview-Ready Answers

Q: How do you connect API Gateway to EKS without ALB?

"I use Istio service mesh. API Gateway handles external concerns like API keys and rate limiting, then routes through a VPC Link to an internal NLB that fronts Istio's Ingress Gateway inside the cluster. Istio replaces ALB by providing load balancing, circuit breaking, canary deployments, automatic mTLS, retries, and timeouts through Envoy sidecars — all without code changes."

Q: What is a service mesh and why use one?

"A service mesh is a dedicated infrastructure layer for service-to-service communication. Istio injects an Envoy sidecar proxy next to every Pod. All traffic flows through these proxies, giving you load balancing, mTLS encryption, circuit breaking, retries, and observability without modifying application code. It is language-agnostic — works the same for Java, Python, Go."

Q: How does Istio canary deployment work?

"Istio VirtualService allows traffic splitting by weight. I deploy v2 alongside v1, then configure 90/10 traffic split. If v2 is healthy, I gradually shift to 80/20, 50/50, then 0/100. If v2 shows errors at any stage, I shift back to 100/0 instantly. This is done by updating one YAML file — no code changes, no redeployment needed."

Q: What is the overhead of Istio?

"Each Pod gets an Envoy sidecar consuming roughly 100m CPU and 50-100 MB memory. The control plane (Istiod) needs about 500m CPU and 1 GB memory. For a 50-Pod cluster, expect approximately 5.5 vCPU and 6 GB extra memory, costing around \$50-100/month. The trade-off is worthwhile for 20+ microservice architectures where mesh benefits outweigh the resource costs."

Q: Istio vs ALB — when to use which?

"ALB is simpler and sufficient for most applications with basic load balancing and routing needs. Istio is for complex microservice architectures needing mTLS, circuit breaking, canary deployments, and distributed tracing without code changes. ALB costs \$20/month; Istio is free but adds CPU/memory overhead. I recommend ALB for under 15 services and Istio for 20+ services or when compliance requires mTLS between all services."